Step 1: Real-World Database Setup (`db.py`)

In production, you want your database to handle dropped connections gracefully and manage connection pools.

Python

```
# db.py
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

DATABASE_URL = "sqlite:///./sqlite.db"
# Note: For real projects, you'd use PostgreSQL:
# DATABASE_URL = "postgresql://user:password@localhost/dbname"

# ==================================================
# Production-Ready Engine
# ==================================================
engine = create_engine(
    DATABASE_URL,
    echo=False, # Turn off in production!
    # pool_pre_ping=True checks if the connection is still alive
    # before using it. Very important for production to prevent timeouts!
    # pool_size=5, max_overflow=10  (Used for Postgres/MySQL, not SQLite)
)

SessionLocal = sessionmaker(
    bind=engine,
    autoflush=False,
    expire_on_commit=False
)
```

Step 2: The "Cheat Sheet" of Column Types & Constraints (`models.py`)

1. All Real-World Column Attributes (Data Types)

This model covers every major data type you will need in a production app.

Python

```
import uuid
import enum
from datetime import datetime, date
from typing import Any
```

```python
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column
from sqlalchemy import (
    String, Text, Integer, BigInteger, Numeric, Boolean,
    DateTime, Date, Enum, JSON, Uuid, func
)

class Base(DeclarativeBase):
    pass

class OrderStatus(enum.Enum):
    PENDING = "pending"
    SHIPPED = "shipped"
    DELIVERED = "delivered"

class ProductData(Base):
    __tablename__ = "product_data"

    # ==================================================
    # 1. PRIMARY KEYS (Real World uses UUIDs, not just Integers)
    # ==================================================
    # UUIDs are harder to guess than 1, 2, 3, making your API much more secure.
    id: Mapped[uuid.UUID] = mapped_column(
        Uuid,
        primary_key=True,
        default=uuid.uuid4 # Auto-generates a UUID in Python when created
    )

    # Standard Integer PK (if you don't want UUID)
    # id: Mapped[int] = mapped_column(primary_key=True)

    # ==================================================
    # 2. STRINGS & TEXT
    # ==================================================
    # String with limit (Good for titles, names, emails)
    title: Mapped[str] = mapped_column(String(100), nullable=False)

    # Text (No limit. Good for descriptions, blog posts, bios)
    description: Mapped[str | None] = mapped_column(Text, nullable=True)

    # ==================================================
    # 3. NUMBERS (Integers & Floats)
    # ==================================================
    # Standard Integer
    stock_count: Mapped[int] = mapped_column(Integer, default=0)
```

```
    # BigInteger (For massive numbers like view counts on a viral video)
    views: Mapped[int] = mapped_column(BigInteger, default=0)

    # Numeric / Decimal (ALWAYS use this for Money, never use Float!)
    # precision=10 (total digits), scale=2 (digits after decimal)
    price: Mapped[float] = mapped_column(Numeric(10, 2), nullable=False)

    # ==================================================
    # 4. BOOLEANS
    # ==================================================
    # server_default sets the default in the actual SQL database, not just Python
    is_active: Mapped[bool] = mapped_column(Boolean, server_default="true")

    # ==================================================
    # 5. DATES & TIMES
    # ==================================================
    # Date only (YYYY-MM-DD)
    release_date: Mapped[date | None] = mapped_column(Date, nullable=True)

    # DateTime with Timezone (Crucial for global apps!)
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now() # Auto-sets current time on creation
    )

    updated_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now(),
        onupdate=func.now() # Auto-updates time whenever the row is modified
    )

    # ==================================================
    # 6. ADVANCED TYPES (Enums & JSON)
    # ==================================================
    # Enum (Restricts input to exact choices defined in Python Enum)
    status: Mapped[OrderStatus] = mapped_column(Enum(OrderStatus),
default=OrderStatus.PENDING)

    # JSON (Perfect for storing flexible settings, features, or unstructured
data)
    # e.g., {"color": "red", "size": "XL", "weight_kg": 1.5}
    metadata_json: Mapped[dict[str, Any] | None] = mapped_column(JSON,
nullable=True)
```

2. All Real-World Constraints

Constraints protect your database from bad data. You can set them on individual columns, or across the whole table.

```
from sqlalchemy import CheckConstraint, UniqueConstraint, Index

class UserAccount(Base):
    __tablename__ = "user_accounts"

    id: Mapped[int] = mapped_column(primary_key=True)

    # ==================================================
    # COLUMN-LEVEL CONSTRAINTS
    # ==================================================

    # 1. UNIQUE: No two rows can have the same email.
    # 2. INDEX: Makes searching by email lightning fast (Crucial for login
queries!).
    email: Mapped[str] = mapped_column(String(255), unique=True, index=True,
nullable=False)

    # 3. NULLABLE=FALSE: This field cannot be empty.
    password_hash: Mapped[str] = mapped_column(String(255), nullable=False)

    age: Mapped[int] = mapped_column(Integer, nullable=False)
    department: Mapped[str] = mapped_column(String(50), nullable=False)
    role: Mapped[str] = mapped_column(String(50), nullable=False)

    # ==================================================
    # TABLE-LEVEL CONSTRAINTS (__table_args__)
    # ==================================================
    # Use this when a constraint depends on multiple columns, or needs custom SQL
rules.
    __table_args__ = (
        # 4. MULTI-COLUMN UNIQUE:
        # A user can only have one specific role per department.
        # e.g., You can't be added as an "Admin" to "HR" twice.
        UniqueConstraint('department', 'role', name='uix_department_role'),

        # 5. CHECK CONSTRAINT:
        # The database literally will reject the save if age is under 18.
        # This is safer than validating in Python, because the DB enforces it
globally.
        CheckConstraint('age >= 18', name='check_user_is_adult'),
```

```
        # 6. MULTI-COLUMN INDEX:
        # If your API frequently searches for "all admins in HR", an index
combining
        # these two columns makes the query massively faster.
        Index('idx_dept_role', 'department', 'role'),
    )
```

Quick Summary of Real-World Rules:

- **Money?** Always use `Numeric`. Never `Float`.
- **Primary Keys?** Start using `Uuid` if you are building a public-facing API.
- **Searching?** Add `index=True` to columns you frequently use in `.where()` clauses.
- **Timestamps?** Always use `server_default=func.now()`. Let the database handle the time, not Python.